

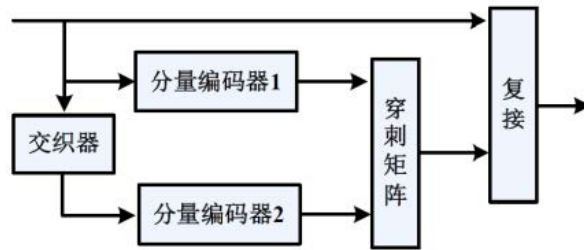
Turbo码

peterrolls@qq.com

2026 年 5 月 30 日

Turbo码是一种并行级联卷积码，其编码器由两个反馈的系统卷积码器通过一个交织器并行连接而成，编码后的校验位经过删余阵，从而产生不同码率的码字。译码时采用迭代译码算法。

主要讨论的是并行级联卷积码(PCCC)编码结构的Turbo码，其结构如图所示：



1 编码

1.1 总码率计算

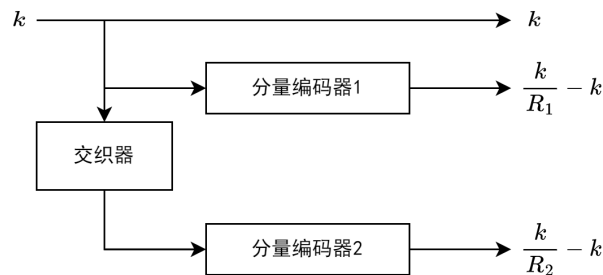
对系统的卷积码 (n_0, k_0) ，设 $R_0 = k_0/n_0$ ，则：

信息符号数： k_0 （每小组）

编码输出符号数： $n_0 = k_0/R_0$

冗余符号数： $n_0 - k_0 = k_0/R_0 - k_0$

对PCCC，设两个分量码的码率分别为 R_1 和 R_2 ：



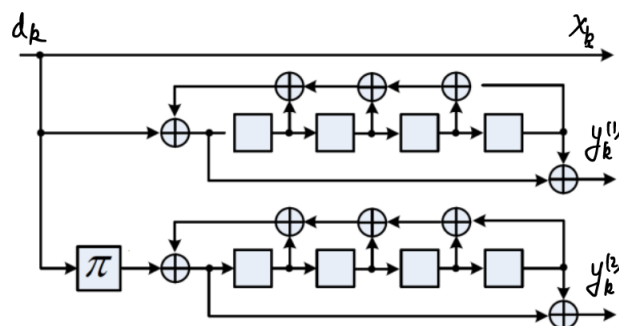
输入信息符号数： k

输出编码符号数： $k + k/R_1 - k + k/R_2 - k$

$$\text{总码率: } R = \frac{k}{\frac{k}{R_1} + \frac{k}{R_2} - k} = \frac{R_1 R_2}{R_1 + R_2 - R_1 R_2}$$

1.2 编码过程

以递归系统(37, 21)码的分量码，图中 π 为交织器：



$$\text{以 } G(D) = \left[1, \frac{1+D^4}{1+D+D^2+D^3+D^4} \right]$$

构建的并行级联卷积码

该编码器的编码效率为 $\frac{1}{3}$ ，经 $P = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 的穿刺（打孔）矩阵去掉一些校验符号，得到码率为0.5的编码。

这里的递归系统码的表示含义如下：

递归系统	(37	21)
二进制表示	011,111	010,001
多项式表示	$D^4 + D^3 + D^2 + D + 1$	$D^4 + 1$

打孔作用：

$$\begin{array}{ccccccc} & x_0 y_0^{(1)} y_0^{(2)} & x_1 y_1^{(1)} y_1^{(2)} & x_2 y_2^{(1)} y_2^{(2)} & x_3 y_3^{(1)} y_3^{(2)} & \cdots \\ \text{打孔} & x \ 1 \ 0 & x \ 0 \ 1 & x \ 1 \ 0 & x \ 0 \ 1 & \\ & x_0 y_0^{(1)} & x_1 y_1^{(2)} & x_2 y_2^{(1)} & x_3 y_3^{(2)} & \cdots \end{array}$$

如此完成编码。

- 为了得到接近Shannon限的系统性能，信息分组长度（交织器大小）一般比较大，通常至少几千个比特。Berrou等在ICC最初的工作中使用了65536的码长（信息长度32768）；
- 对于分量码来说，一般选择相同结构，且约束长度较短，通常小于4；
- 递归分量码（由系统反馈编码器产生）会比非递归分量码有更好的性能；
- 高码率可通过穿刺（打孔）矩阵产生；
- 通过增加分量码和交织器也可以得到较低编码速率的Turbo码；
- 最好的交织器能够对比特以伪随机的方式进行排序，传统的块交织器（行——列）在Turbo码中性能不好，除非块长度很短；
- 由于交织器只是对比特位置进行重新排序，因此，交织后的序列与原始序列具有相同的重量；
- 对每个分量码来说，用BCJR（或MAP）算法作为SISO（软输入软输出）译码器能够获得最好的性能。因为MAP译码器使用了前向——后向算法，信息是以块的形式进行的，对未使用交织器的分量译码器来说，附加 v 个0比特能够让它返回到全0态，有交织器作用的不行。

Turbo码有两大缺点:

1. **较大的译码时延**, 这是由于码长度较大, 译码需要多次迭代造成的。这样对于实时业务或高速数据的传输就非常不利;
2. BER在 10^{-5} 后会出现**误码平台**, 这是由于Turbo码的重量分布造成的。可以考虑采用SCCC (串行级联卷积码) 结构的卷积码降低错误平台。

2 最大后验译码算法

设每个RSC (递归系统码) 分量码约束长度为 K , 在时刻 k , 编码器状态 S_k 设为:

$$S_k = (a_k, a_{k-1}, \dots, a_{k-K+1})$$

另外, 假设信息序列 $\{d_k\}$ 是 N 个取自 $\{0, 1\}$ 的独立等概二进制随机变量组成, 编码器的初值和终值为零:

$$S_0 = S_N = \underbrace{(0, 0, \dots, 0)}_{K \text{ 个}} = \vec{0}_{K \times 1}$$

设编码器输出码字序列 $C_1^N = \{C_1, \dots, C_k, \dots, C_N\}$, 送入离散高斯无记忆信道, 信道输出为 $R_1^N = \{R_1, \dots, R_k, \dots, R_N\}$, 这里 $R_k = (x_k, y_k)$ 。

设有联合概率为:

$$\lambda_k^{(i)}(m) = P\{d_k = i, s_k = m | R_1^N\}$$

则解码比特 d_k 的后验概率为:

$$P\{d_k = i | R_1^N\} = \underbrace{\sum_m P\{d_k = i, s_k = m | R_1^N\}}_{\text{全概率公式}} = \sum_m \lambda_k^{(i)}(m), i = 0, 1$$

这样, 可以得到解码符号 d_k 的对数似然比, 然后进行判决输出即可。

以下是解码步骤。

第0步: 初始化概率值 $\alpha_0^{(i)}(m)$ 及 $\beta_N(m)$ 。

$$\alpha_0^{(i)}(m) = \begin{cases} 1 & \text{若 } m = 0, i = 0, 1 \\ 0 & \text{若 } m \neq 0 \end{cases}$$

$$\beta_N(m) = \begin{cases} 1 & \text{若 } m = 0 \\ 0 & \text{若 } m \neq 0 \end{cases}$$

第1步: 对于每一个 $R_k = (x_k, y_k)$ 配对, 利用下面的公式计算 $\alpha_k^{(i)}(m)$ 和 $\beta_k(m)$:

$$\alpha_k^{(i)}(m) = \frac{\sum_{m'} \sum_{j=0}^1 r_i(R_k, m', m) \alpha_{k-1}^{(i)}(m')}{\sum_m \sum_{m'} \sum_{s=0}^1 \sum_{t=0}^1 r_s(R_k, m', m) \alpha_{k-1}^{(t)}(m')}$$

$$r_i(R_k, m', m) = P\{R_k | d_k = i, s_k = m, s_{k-1} = m'\} P\{d_k = i | s_{k-1} = m', s_k = m\} P\{s_k = m | s_{k-1} = m'\}$$

第2步: 当序列 R_1^N 完全收到之后, 利用下面的式子计算每一个 $\beta_k(m)$:

$$\beta_k(m) = \frac{\sum_{m'} \sum_i r_i(R_{k+1}, m, m') \beta_{k+1}(m')}{\sum_m \sum_{m'} \sum_{i=0}^1 \sum_{j=0}^1 r_i(R_{k+1}, m', m) \alpha_i^{(j)}(m')}$$

并计算联合概率 $\lambda_k^{(i)}(m) = \alpha_k^{(i)}(m) \beta_k(m)$ 。

事实上，由前面的推导有：

$$\Lambda(d_k) = \log \frac{\sum_m \lambda_k^{(1)}(m)}{\sum_m \lambda_k^{(0)}(m)} = \log \underbrace{\frac{P(x_k|d_k=1)}{P(x_k|d_k=0)}}_{\frac{2}{\sigma_z^2} Ax_k} + \log \underbrace{\frac{\sum_m \sum_{m'} \sum_{j=0}^1 r_1(y_k, m', m) \alpha_{k-1}^{(j)}(m') \beta_k(m)}{\sum_m \sum_{m'} \sum_{j=0}^1 r_0(y_k, m', m) \alpha_{k-1}^{(j)}(m') \beta_k(m)}}_{\text{令为 } w_k}$$

这里设发送 $d_k = +1$ ($d_k = -1$) 时 x_k 是均值为 A ($-A$) 的高斯变量 $\frac{2}{\sigma_z^2} Ax_k$ ，即：

$$\Lambda(d_k) = \frac{2}{\sigma_z^2} Ax_k + w_k$$

其中 $w_k = \Lambda(d_k)|_{x_k=0}$ 就是编码器提供的冗余信息。

注意 w_k 所代表的“外信息”与解码器输入 x_k 无关。该特性可用来设计并行级联结构编码器的译码器。

3.2 管线结构解码器

反馈译码器可用管线结构解码器实现，其结构如下：

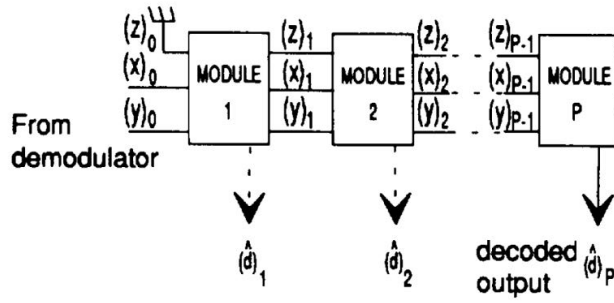


图 3: 管道结构解码器

第一级译码器有3个输入 (x_k, y_k, z_k) ，其概率 $\alpha_{1k}^{(i)}(m)$ 和 $\beta_{k1}(m)$ 基于前面的 $\alpha_k^{(i)}(m)$ 和 $\beta_k(m)$ 进行计算，但将 $R_k = (x_k, y_{1k})$ 用 $R_k = (x_k, y_{1k}, z_k)$ 代替。其 z_k 可假设为高斯变量，方差为 σ_z^2 ：

$$P\{R_k|d_k = i, s_k = m, s_{k-1} = m'\} = P(x_k|\cdot)P(y_{k1}|\cdot)P(z_k|\cdot)$$

这样其 DEC_1 的输出 LLR：

$$\Lambda(d_k) = \frac{2}{\sigma_z^2} Ax_k + \frac{2}{\sigma_z^2} A_z z_k + w_{1k}$$

其中 A 为发送符号“+1”时 x_k 的均值， A_z 为发送符号“+1”时 z_k 的均值， w_{1k} 是 DEC_1 提供的外信息，与序列 $\{z_n\}_{n \neq k}$ 有关。

注意 z_k 是 DEC_2 上一次迭代的输出信息，不应在新一轮迭代中在此输入到 DEC_2 解码器。

因此，对于迭代轮次 p ($p \geq 2$)， DEC_2 的输入包括序列 $\hat{\Lambda}_1(d_n)$ 和 y_{2k} 。其中：

$$\hat{\Lambda}_1(d_n) = \Lambda_1(d_n)|_{z_n=0}$$

DEC_1 使用 $R_k = (x_k, y_{1k}, z_k)$ 作为输入，利用 BCJR 译出 $\text{LLR} \hat{\Lambda}_1(d_n)$ ，然后减去 $\frac{2}{\sigma_z^2} A_z z_k$ 。

DEC_2 输出的外信息：

$$w_{2k} = \Lambda_2(d_k)|_{\hat{\Lambda}_1(d_k)=0}$$

DEC_2 使用 $\hat{\Lambda}_1(d_n)$ 及 y_{2k} 作为输入，利用 BCJR 译出 $\text{LLR} \Lambda_2(d_k)$ ，然后消去 $\hat{\Lambda}_1(d_k)$ 的影响。

DEC_2 输出的似然比 $\Lambda_2(d_k)$ 可送入判决器，输出译码结果 $\hat{d}_k = \text{sgn}[\Lambda_2(d_k)]$ 。

译码过程中，外信息 z_k 的方差 σ_z^2 以及 DEC_2 的输入序列 $\hat{\Lambda}_1(d_k)$ 的方差需要在每一迭代轮 p 重新估计。

其他的一些译码算法：Log-MAP，Max-Log-MAP，SOVA 等；

译码性能：MAP 算法最好，其次是 Log-MAP，然后是 Max-Log-MAP 和 SOVA；

复杂度：Max-Log-MAP 最低，其次是 Log-MAP 和 SOVA，最后是 MAP。